



# OpenArm 小白指南

具身智能的"Linux 时刻"——日本 enactic 全开源 7-DOF 人形双臂平台全拆解

开源硬件 CERN-OHL-S

软件全 Apache-2.0

9 个子仓库

无硬件可玩 4 实验

来源: X @AYi\_AInotes 推文 (2026-09) | GitHub: enactic/openarm | 文档: docs.openarm.dev

产出: F:\reminder\openarm (含 exercise 4 个真跑通实验) | 2026-10-03

Q1 是什么

Q2 Linux时刻

Q3 QDD反驱

Q4 重力补偿

Q5 主从遥操作

Q6 CAN/MIT帧

Q7 7DOF布局

Q8 MJCF解剖

Q9 数据集

Q10 Sim2Real

Q11 开源许可

Q12 怎么上手

## Q1 OpenArm 到底是什么？开源了哪些东西？

一句话：**一套 7 自由度、按人体比例 (160~165cm 人群) 设计的开源机械臂**，东京机器人公司 enactic 把从 CAD 图纸到电机固件到控制代码的全部家底都放上了 GitHub，整机约 \$6,500、双臂 14-DOF 约 \$13,000，负载约 6kg。

它不是"一个仓库"，而是 **1 个门户 + 9 个子仓库** 的家族（实测盘点见实验①）：

| 子仓库                              | 装的是什么                                                                        | 许可证                  |
|----------------------------------|------------------------------------------------------------------------------|----------------------|
| openarm_hardware                 | CAD 图纸 (STEP 全装配/STL 可打印) + BOM + 接线图 (实体放 Google Drive, GitHub Releases 镜像) | <b>CERN-OHL-S v2</b> |
| openarm_can                      | C++17 CAN 总线电机控制库 (达妙电机)                                                     | Apache-2.0           |
| openarm_mujoco                   | MuJoCo 模型 v0.3/v1/v2 + 浏览器 WASM 版                                            | Apache-2.0           |
| openarm_description              | URDF/xacro 机器人描述 (ROS 用)                                                     | Apache-2.0           |
| openarm_ros2 / openarm_isaac_lab | ROS2 集成 / Isaac Lab 训练环境                                                     | Apache-2.0           |
| openarm_teleop                   | 主从遥操作 (单边/双边控制)                                                              | Apache-2.0           |
| openarm_dataset / dora-openarm   | 数据采集 (LeRobot 格式) / Dora 数据流节点                                               | Apache-2.0           |

还有个理念产品 **OpenArm Cell**：统一背景、光照、相机的标准化评测台，让全球研究者的数据**在同一条生产线上产出**。

## Q2 为什么叫具身智能的"Linux 时刻"?

类比两条历史：1991 年 Linux 把只有大厂玩得起的 Unix 服务器系统搬上每个人的电脑；2005 年 RepRap 把工业 3D 打印机拆成 self 复制的开源套件。共同点：

过去：封闭力控双臂  
**\$数万 ~ \$10万+**

大厂专属 · 算法被硬件锁死



现在：OpenArm 全开源  
**\$6,500 / 单臂**

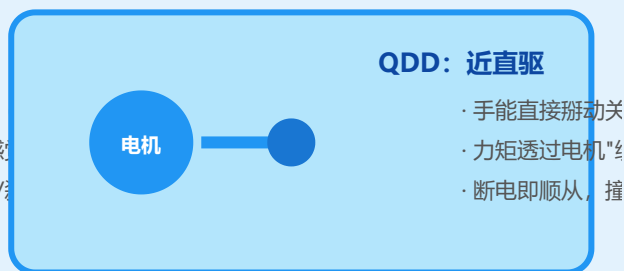
图纸+BOM+固件+代码全拿走

研究者只管  
算法层创新

硬件被开源降维成"平民基建"后，竞争的赛道才转移到算法和数据上——这就是"时刻"的含义。

## Q3 QDD 准直驱是什么？为什么"高反驱" = 安全？

QDD (Quasi-Direct Drive, 准直驱)：电机几乎不经大减速比齿轮箱、直接驱动关节（小比减速或皮带一级）。对比传统工业臂的大减速比谐波箱：



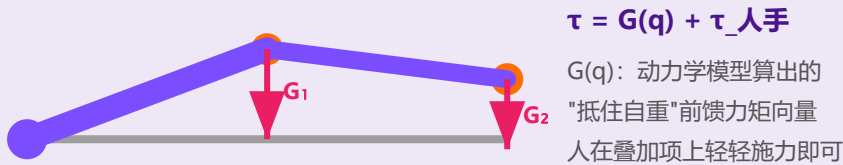
反驱性 (Backdrivability) = 你从输出端施力，电机端会被带着转。**反驱越好，电机绕组切割磁感线产生的反电动势就天然把碰撞力"报"给控制器**——不需要额外六维力传感器就获得全身触觉。这就是 OpenArm 能"用手拖拽示教"的物理根基。

类比：[[智能舵机与菊花链总线]] 里的舵机是"内置减速箱的哑巴+工号报数"；QDD 电机反过来，**减掉减速箱换取力觉透明**。

## Q4 重力补偿是怎么让机械臂"失重"的？

高反驱是把双刃剑：**断电的 QDD 手臂会被自己的重量拖着塌下去**。我的仿真实验A实测（MuJoCo, v1 模型，总质量 5.39kg）：力矩全零 2 秒，肩关节 joint2 从  $0^\circ$  掉了  **$100.1^\circ$** ，峰值角速度 7.64 rad/s。

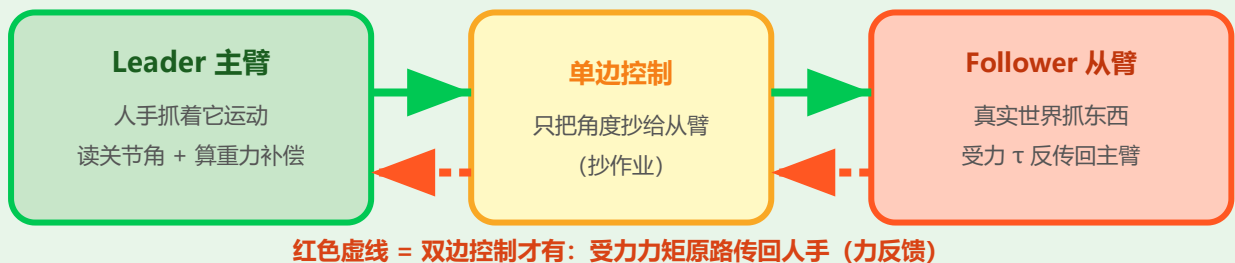
解法 = **重力补偿 (Gravity Compensation)** 前馈：按当前位姿，用动力学模型算出"每个关节恰好抵住自重所需的力矩  $\tau_g$ "，先发给电机。剩下的才是人手/算法要出的力——于是手臂摸起来像**失重的漂浮状态**，拖拽示教毫不费力。



实验B 实测对照：竖直伸展位（重力矩最大的位姿）保持 3 秒——**纯 PD 反馈 joint2 残差  $0.051^\circ$ 、joint6 残差  $0.135^\circ$ ；加上重力补偿前馈后 7 个关节误差全部归零（改善 100%）**。前馈管"已知"，反馈管"意外"，这就是控制里的黄金搭档。

## Q5 主从遥操作：单边 vs 双边（力反馈）差在哪？

OpenArm 用一台**同款手臂当主控手柄 (Leader Arm)**，与从臂 (Follower) 1:1 跟随，leader 侧控制环跑 500Hz。



**单边 (unilateral)**：从臂抄主臂角度，抓没抓到东西，人手毫无感觉。**双边 (bilateral)**：从臂每个关节受到的外力矩，经同样"高反驱"的主臂转成真实的推拉感——**捏鸡蛋的力度感直接回到你掌心**。正因主从臂是同一套 QDD 硬件，力才"透得回来"；这也解释了为什么主臂叫 **KER (Kinematic EXO-frame Replica, 外骨骼式复刻架)** 架构。

teleop 仓库里正是三个控制器文件：gravity\_compasation / unilateral\_control / bilateral\_control (C++, 500Hz 定时线程)。

## Q6 一条 CAN 线怎么指挥 7 个电机？MIT 帧的 8 字节里藏了什么？

OpenArm 每个关节一颗**达妙 (Damiao) QDD 电机**，全部挂在一条 **CAN 总线** 上（2 线差分，[[智能舵机与菊花链总线]] 的“一根线串一串”思想，只是协议换成汽车级的 CAN）。控制库 openarm\_can 支持 13 种达妙型号、4 种控制模式（MIT / POS\_VEL / VEL / POS\_FORCE）。

最核心的是 **MIT 模式**（源自 MIT 猎豹机器人约定）：一条 8 字节帧塞下 5 个控制量——

$$8 \text{ 字节} = 16\text{bit } q + 12\text{bit } dq + 12\text{bit } kp + 12\text{bit } kd + 12\text{bit } \tau = 64\text{bit 满排}$$

|      |      |       |           |       |       |            |       |
|------|------|-------|-----------|-------|-------|------------|-------|
| q 高8 | q 低8 | dq 高8 | dq低4 kp高4 | kp 低8 | kd 高8 | kd低4 tau高4 | tau低8 |
|------|------|-------|-----------|-------|-------|------------|-------|

位置 16bit

速度/刚度/阻尼/力矩 各 12bit 定点压缩

**定点映射：** $u = (x - \min) / (\max - \min) \times (2^{\text{bits}} - 1)$  （DM4310:  $q \in \pm 12.5\text{rad}$ ,  $dq \in \pm 30\text{rad/s}$ ,  $\tau \in \pm 10\text{N}\cdot\text{m}$ ）  
反馈帧另有布局：D0=电机ID|错误码<4, q16|dq12|tau12 + 两路温度

实验③我把它用 Python 逐位复刻并与手算对照： $q=1.0\text{rad} \rightarrow 0x8A3D$  完全一致，1000 组随机值编解码往返误差全部  $\leq$  半个分辨率（0.00019 rad）。所谓“电机协议”不过是一张**定点压缩的位排布表**——拆开就不再神秘。9 种错误码（过压/欠压/过流/过热/失联/过载...）藏在反馈帧 D0 高 4 位，电机指示灯就靠它闪。

## Q7 7 自由度怎么布局？为什么不是 6 个？

人臂就是 7 DOF：肩 3 + 肘 1 + 腕 3。OpenArm 的 MJCF 实测关节表（含限位）：

| 关节       | 轴         | 限位                                           | 类比人体                |
|----------|-----------|----------------------------------------------|---------------------|
| joint1   | Z（竖直）     | $-80^\circ \sim +200^\circ$                  | 肩：内外旋               |
| joint2   | X         | $-100^\circ \sim +100^\circ$                 | 肩：前后摆               |
| joint3   | Z         | $\pm 90^\circ$                               | 肩：侧摆                |
| joint4   | Y         | $0^\circ \sim +140^\circ$                    | 肘                   |
| joint5~7 | Z/X/Y     | $\pm 90^\circ / \pm 45^\circ / \pm 90^\circ$ | 腕：3 轴球窝             |
| finger×2 | slide（平移） | $0 \sim 44\text{mm}$                         | 平行夹指（equality 耦合同步） |

**为什么 7 > 6？** 6 个自由度只够把末端摆到“任意位置+任意姿态”，但 7 DOF 多出一个**冗余自由度：末端定住不动，手臂还能在内部“换姿势”**（自运动）——绕开桌腿、避开障碍、让肘更贴近身体，像人手肘不动也能拧钥匙一样。代价是逆运动学从“唯一解”变成“无穷解”，要靠优化挑最舒服的那个（见 [[逆运动学 IK 与 CCD 迭代]]）。链条与 [[前向运动学 FK 与 四元数链]] 完全同构：7 个 hinge + 2 个 slide 手指沿运动链折叠，实验②里我摆出  $\text{joint2}=90^\circ + \text{joint4}=90^\circ$  的抓取姿，末端位移 683mm，与模型几何吻合。

## Q8 MuJoCo 模型文件里，机械臂长什么样？

openarm\_mujoco 提供 v0.3 / v1 / v2 三代 MJCF + 浏览器 WASM 版。v1 单臂实测：**nq=9, nv=9, 执行器 8 个, 刚体 10 个, 总质量 5.390kg, dt=2ms**。文件结构四件套：

```
default    → 类"CSS": class="motor" 统一 damping=0.25, frictionloss=0.02
worldbody  → link1→...→link7→hand→finger×2 严格嵌套（[[Three.js与场景图]] 同款父子树）
tendon     → 固定滑绳 0.5×finger1 + 0.5×finger2
equality   → 把两根手指焊成"同步开合"约束
actuator   → 8 个 motor（joint2 特例 ctrlrange=±12, 注释：克服全伸展重力）
```

两个值得学的细节：① 手指不是双执行器——**1 个执行器 + equality 约束让两指镜像同步**，省钱又天然同步；② **visual 与 collision 网格分离**（视觉用精细 OBJ、碰撞用粗包络 STL），渲染好看、碰撞便宜——所有物理引擎的通用手法。

实验②离屏渲染出机械臂结构剪影（exercise/ex2\_render.png）；断电实验证明这个模型"物理是活的"：不给力矩它就诚实地塌下去。

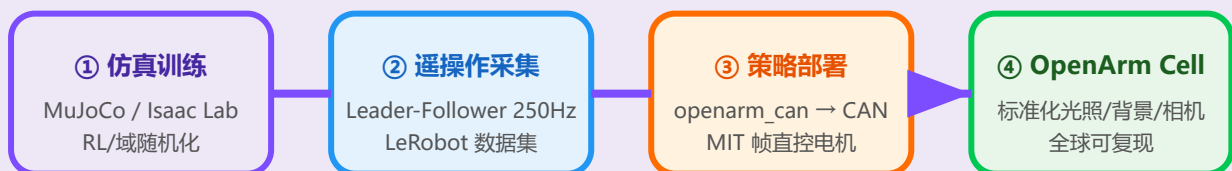
## Q9 遥操作采下来的数据长什么样？（LeRobot 格式）

模仿学习的"燃料"就是遥操作数据。openarm\_dataset 采用 Hugging Face **LeRobot v2.1/v3.0 格式**（parquet + 视频）。官方自带的测试数据实测（实验④）：

```
episodes: 2 条（1 成功 1 失败——失败数据也留档！）
观测 7 键: arms/{right,left}/{qpos,qvel,qtorque} 各 (746, 8) + lifter/elevation
动作键:    arms/{right,left}/qpos + lifter/elevation
列名:      joint1..joint7 + gripper, 时间戳精度到纳秒, 帧间隔 4.0ms = 250Hz
```

三个要点：① **8 列 = 7 关节 + 夹爪**，和 [[策略观测向量与动作契约]] 的"逐槽对齐"契约一模一样；② **观测里带 qtorque (力矩)**——这是 **QDD 免费力传感器的红利**，普通舵机臂想记都记不了；③ 连"失败 episode"都保留，因为模仿学习恰恰需要负样本。

## Q10 Sim-to-Real 路线：从 MuJoCo 到真机怎么走？



同一套 URDF/MJCF 描述贯穿四站——"模型即合同"，sim 与 real 看到的是同一具身体

与 [[Sim2Real与MuJoCo仿真]]（microduck）同一条路，但 OpenArm 把"身体描述"标准化成公共基建：MJCF/URDF/Isaac 三格式同源维护，仿真里调好的策略，参数语义与真机 CAN 指令——对应。

# Q11 CERN-OHL-S 和 Apache-2.0 有什么区别？

|       | Apache-2.0 (软件仓库) | CERN-OHL-S v2 (硬件 CAD)                                               |
|-------|-------------------|----------------------------------------------------------------------|
| 管的对象  | 代码                | 物理设计文件 (STEP/STL/BOM)                                                |
| 传染性   | 宽松：可闭源二进制分发       | <b>强互惠 (S=Strongly Reciprocal)</b> ：改了硬件设计并"对外分发" (卖/发) 成品，设计文件也必须开源 |
| 类比    | MIT/GPL 之间的宽松派    | 硬件界的 GPL                                                             |
| 对你的意思 | 随便改随便商用           | 自用随意； <b>想拿它做闭源商业臂？门都没有</b>                                          |

这套"软件宽松 + 硬件强互惠"的双许可组合是开源硬件项目的标准操作：代码放开吸引生态，图纸用 OHL-S 防止白嫖成品闭源售卖。

# Q12 我没有 \$6,500 的硬件，能拿 OpenArm 玩什么？

本次四个真实跑通的实验就是答案（全部 0 硬件、0 费用，Windows 直接跑）：

| 实验         | 做了什么                                             | 硬核产出                             |
|------------|--------------------------------------------------|----------------------------------|
| ①仓库盘点      | 脚本统计 1 门户 + 6 子仓库的文件/体积/许可证                      | 开源范围一张表（发现 CAD 实体在 Google Drive） |
| ②MuJoCo 仿真 | 加载 v1 MJCF → FK 摆位 → 断电下坠 → PD vs PD+重力补偿 → 离屏渲染 | 重力补偿让 7 关节保持误差改善 <b>100%</b>     |
| ③MIT 协议复刻  | Python 逐位实现定点编解码 + 1000 组 roundtrip + 错误码解剖      | 4/4 PASS，误差 ≤ 半分辨率               |
| ④真实数据集     | pip 装 openarm_dataset 读官方采样数据                    | 250Hz×8 关节契约、含 qtorque、含失败样本     |

再进阶的路线：改 MJCF 换关节限位重新仿真 → 跑 Isaac Lab 强化学习任务 → 用 dora-openarm 接自己的输入设备做"伪主臂"（键盘/手柄替 \$6,500 的 Leader）。

## 🎁 这个项目给我什么（能力清单）

- ① **一套能整体照抄的开源机器人工程范式**：描述层（MJCF/URDF）↔ 通信层（CAN/MIT 帧）↔ 控制层（重力补偿/双边遥操作）↔ 数据层（LeRobot）各就各位，任何机器人项目都长这样；
- ② **QDD 力控思路**：想给任何电机加“力觉”，先看反驱性 + 学重力补偿前馈（实验②的 100% 改善可直接复用到别的臂）；
- ③ **MIT 定点帧格式**：嵌入式与电机对话的通用语法（达妙/宇树/小米电机都兼容此约定）；
- ④ **进入具身智能研究的最低成本门票**：MuJoCo 免装浏览器版 + 官方数据集，今天就能开始；
- ⑤ **开源硬件双许可示范**：自己以后开源硬件项目照此搭配；
- ⑥ 与已学项目互补：**microduck 是腿（足式 RL）**，**reachy-mini 是脸（表情桌面机器人）**，**OpenArm 是手（力操作双臂）**——三条线合起来就是具身智能的“身·脸·手”全家福。

## 🔧 动手实验实录（4 个实验全部真实运行，无硬件诚实记录）

**实验① 仓库盘点**：python exercise/ex1\_repo\_survey.py → 输出 7 行体积/许可表。✅ 预期结果：openarm\_hardware 行显示仅 10 文件（CAD 指针仓库，实体在 Google Drive, CERN-OHL-S），其余子仓库全 Apache-2.0。

**实验② MuJoCo 仿真**：python exercise/ex2\_mujoco\_arm\_sim.py → 模型解剖 + FK + 两个对照子实验。  
✅ 预期结果：断电 2s joint2 掉 100.1°；竖直伸展保持 3s，纯 PD 残差最大 0.135°，加重力补偿后全 0；side 视角结构剪影存 ex2\_render.png。

**实验③ 协议复刻**：python exercise/ex3\_mit\_protocol.py。✅ 预期结果：8 字节帧 8A 3D 77 73 33 4C CB FF 手算一致；1000 组往返误差 ≤ 半分辨率；4/4 组检查全 PASS。

**实验④ 数据集**：python exercise/ex4\_dataset\_load.py。✅ 预期结果：episode 0 观测 7 键、右臂 qpos (746,8)、帧间隔 4.0ms (250Hz)、8 维全程行程 0.46~0.55 rad。

⚠️ 诚实记录：所有结论来自仿真与官方离线数据，未接触真实硬件——反驱手感、CAN 时序、电机噪声均未验证；实验②渲染因金属材质无环境贴图偏暗，仅作结构剪影；本版 openarm\_dataset 的 load\_obs 需传 episode dict (README 的 load\_obs(0) 是旧签名)。